

Document 1 — Training the Shadow-Invariant Vision Descriptor

SIH26166 · RTX 6000 Pro workflow via the ARC-AGI-3 Kaggle notebook, and integration into the registration pipeline

Updated August 31, 2026 — merges Document 7 (Kaggle Training Workflow, Elaborated Setup & Dataset Checklist) into this document. Document 7 is now retired as a standalone file; everything it added lives in Part A below.

What changed in this update: Document 7 confirmed that training on a generic, non-competition Kaggle notebook (full internet, T4×2/P100) and then attaching the exported ONNX weights as a Dataset input to the ARC-AGI-3 notebook is a legitimate, commonly-used pattern — and is the recommended default path for this project's small PatchEncoder model. That confirmed workflow, its full six-cell script, and the 20 GB dataset-size discipline are now folded into Part A as A.3 (Path 1, original) is kept, a new A.4 (Path 2, generic notebook, confirmed) is added, and A.5 states which one to use.

Contents

- Part A — The ARC-AGI-3 Kaggle notebook, and how to actually use it
 - A.1 What ARC-AGI-3 / the Kaggle kernel actually is
 - A.2 Using the ARC-AGI-3 notebook directly, and the one constraint that actually matters
 - A.3 Path 1 — full setup inside the ARC-AGI-3 notebook, rtx6000 accelerator
 - A.4 Path 2 — generic Kaggle notebook (T4×2), confirmed working, full script
 - A.5 Which path to use
 - A.6 Keeping every output under Kaggle's 20 GB cap
- Part B — Real training infrastructure options, ranked by what's actually available to you
- Part C — What the descriptor model actually does in your pipeline

Part A — The ARC-AGI-3 Kaggle notebook, and how to actually use it

A.1 What ARC-AGI-3 / the Kaggle kernel actually is

ARC-AGI-3 is the third generation of the Abstraction and Reasoning Corpus benchmark from the ARC Prize Foundation, run as ARC Prize 2026 on Kaggle. It is an interactive reasoning benchmark: an agent is dropped into 110 unseen 64×64-grid game environments with no instructions and no natural-language interface, and must learn the mechanics, goal, and win-condition purely through interaction. It is judged on general adaptability across unseen environments, not on any single trained skill.

The Kaggle competition infrastructure, as documented in the competition's own docs and CLI guide:

- Kaggle provisions an RTX Pro 6000 GPU with 96 GB of VRAM for competition notebooks.
- Session/runtime limits are stated at roughly 9 hours per run in the preview-period discussion (some write-ups round this to “up to 12 hours” — check the live rules page for the current number).
- The accelerator is opt-in per notebook: default builds against a T4 GPU; a competitor edits `ACCELERATOR = "rtx6000"` in `scripts/build_notebook.py` to request the RTX 6000 Pro tier. The docs are explicit: “RTX 6000 is reserved for ARC-AGI-3 notebooks — don't use it for early iteration.”

- All accelerated Kaggle sessions for this competition run with internet access disabled — stated directly in the competition kit documentation.
- Prize-eligible submissions must be released under a permissive/public-domain license (CC0 or MIT-0) and are evaluated in a sandboxed environment.

A.2 Using the ARC-AGI-3 notebook directly, and the one constraint that actually matters

A code-competition notebook is a normal interactive Kaggle editor: once you've accepted the competition rules, you can select the rtx6000 accelerator in the notebook's Settings panel and run whatever cells you write there. Nothing about the editor itself restricts you to the competition's own agent-evaluation code — the “don't use it for early iteration” line is guidance, not a technical lock. Only the separate “submit to competition” action actually enters anything into ARC Prize 2026.

The one constraint that's real regardless of how you use the notebook: all accelerated Kaggle sessions (rtx6000 included) run with internet access disabled. Your training data lives at chmapbrowse.issdc.gov.in, pradan.issdc.gov.in, lroc.im-ldi.com, and the JAXA SELENE archive — a no-internet session can't pull any of that mid-run. Do data prep in a step where you have internet, attach the result as a Kaggle Dataset, then switch to the accelerated session for the GPU-heavy part.

A.3 Path 1 — full setup inside the ARC-AGI-3 notebook, rtx6000 accelerator

This is the original path: data prep in an internet-on pass, then the same competition notebook switches to the rtx6000 accelerator for training.

1. Get in. kaggle.com/competitions/arc-prize-2026-arc-agi-3 → accept the competition rules (this unlocks the accelerator option) → Create → New Notebook, or fork github.com/arcprize/ARC-AGI-3-Kaggle-Starter.

2. Data prep pass — internet ON, no accelerator needed. PRADAN/Chandrayaan Map Browse/LROC/JAXA are gated portals; no accelerated session will reach them mid-run, so this happens first, separately (see Document 3 for the full acquisition sequence). Crop to patches and generate synthetic pairs at this stage too — upload only the small cropped output, never full scenes, to stay well inside Kaggle's 20 GB private-dataset cap:

```
zip -r lunar_patches_v1.zip data/dem_patches/ data/synthetic_pairs/
kaggle datasets create -p ./lunar_patches_v1 --dir-mode zip
```

3. Switch to the rtx6000 session for training. Notebook Settings → Accelerator → the rtx6000 tier. Add Input → your lunar-patches-v1 dataset → mounts read-only at `/kaggle/input/lunar-patches-v1/`.

```
# Cell 1 -- confirm the accelerator actually came up
import torch
print(torch.cuda.is_available(), torch.cuda.get_device_name(0))

# Cell 2 -- same PatchEncoder / triplet_loss training loop as Part C.1-C.2,
# pointed at the mounted dataset
import torch, torch.nn as nn, torch.nn.functional as F, os

DATA_DIR = "/kaggle/input/lunar-patches-v1"
CKPT_DIR = "/kaggle/working/checkpoints"
os.makedirs(CKPT_DIR, exist_ok=True)

class PatchEncoder(nn.Module):
    def __init__(self, embed_dim=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 32, 5, stride=2, padding=2), nn.BatchNorm2d(32), nn.ReLU(),
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=2, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.AdaptiveAvgPool2d(1),
```

```

    )
    self.fc = nn.Linear(128, embed_dim)

    def forward(self, x):
        z = self.fc(self.net(x).flatten(1))
        return F.normalize(z, dim=1)

def triplet_loss(a, p, n, margin=0.3):
    d_pos = 1 - (a * p).sum(dim=1)
    d_neg = 1 - (a * n).sum(dim=1)
    return F.relu(d_pos - d_neg + margin).mean()

model = PatchEncoder().cuda()
opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=50)

for epoch in range(50):
    for anchor, positive, negative in dataloader:
        anchor, positive, negative = anchor.cuda(), positive.cuda(), negative.cuda()
        with torch.autocast("cuda", dtype=torch.bfloat16):
            loss = triplet_loss(model(anchor), model(positive), model(negative))
        opt.zero_grad(); loss.backward(); opt.step()
    sched.step()
    if epoch % 10 == 0:
        torch.save(model.state_dict(), f"{CKPT_DIR}/epoch{epoch}.pt")

# Cell 3 -- export; keep /kaggle/working small regardless of which cap you're near
model.eval()
dummy = torch.randn(1, 1, 128, 128, device="cuda")
torch.onnx.export(model, dummy, "/kaggle/working/descriptor.onnx",
    input_names=["patch"], output_names=["embedding"], opset_version=17)

```

4. Pull it out. Commit, Output → Download descriptor.onnx (a few MB), drop into models/ per §C.4–C.5. Clear old epoch checkpoints from /kaggle/working before committing. Session budget is the ~9-hour figure against a ~30 GPU-hour/week account quota (verify current numbers — they move) — comfortable headroom for 50 epochs over patch-sized triplets. The only hard sequencing rule: Step 2 before Step 3, no internet after the accelerator is on.

A.4 Path 2 — generic Kaggle notebook (T4×2), confirmed working, full script

Confirmed: opening a standard, non-competition-scoped Kaggle notebook, doing all data prep and training there with full internet access, exporting to descriptor.onnx, and attaching that export as a Dataset input to the ARC-AGI-3 notebook is a legitimate, commonly-used Kaggle pattern. This is the confirmed default path for this project's PatchEncoder model, which is small enough that the RTX 6000's extra VRAM buys nothing — T4×2 trains it in minutes.

The workflow, precisely:

- Open a standard, non-competition-scoped Kaggle notebook (Settings → Accelerator → GPU T4 x2, or P100). These have full internet access and are not subject to the ARC-AGI-3 no-internet rule.
- Do data prep and model training there: pull PRADAN/LROC/DEM data, generate synthetic pairs, train PatchEncoder, export to descriptor.onnx.
- Package the exported artifact (a few MB, not the raw imagery) as a new Kaggle Dataset: `kaggle datasets create`.
- Open the ARC-AGI-3 competition notebook, Add Input → your dataset. It mounts read-only at /kaggle/input/<dataset-slug>/, exactly like any other Kaggle Dataset attached to any notebook.
- Load the ONNX weights with onnxruntime inside the ARC-AGI-3 notebook and run inference / further pipeline code against the rtx6000 accelerator, using the dataset as a pre-trained artifact rather than training from scratch inside that notebook.

A.4.1 Notebook settings

- kaggle.com → Create → New Notebook (do not fork from a competition — keeps it outside the ARC-AGI-3 competition scope entirely).
- Notebook Settings → Accelerator → GPU T4 x2 (or P100 if offered). Internet: On.
- Persistence: “Files only” is fine — the ONNX file is exported at the end and survives a session restart via Output.

A.4.2 Cell 1 — install & imports

```
!pip install -q rasterio onnxruntime pds4-tools
import os, glob, random
import numpy as np
import rasterio
import torch, torch.nn as nn, torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader

DATA_DIR = "/kaggle/working/data"
DEM_DIR = f"{DATA_DIR}/dem_patches"          # populated in Cell 2
PAIR_DIR = f"{DATA_DIR}/synthetic_pairs"     # populated in Cell 3
CKPT_DIR = "/kaggle/working/checkpoints"
os.makedirs(CKPT_DIR, exist_ok=True)
os.makedirs(PAIR_DIR, exist_ok=True)

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(torch.cuda.is_available(), torch.cuda.device_count(),
      [torch.cuda.get_device_name(i) for i in range(torch.cuda.device_count())])
```

A.4.3 Cell 2 — attach your DEM patches (uploaded as a Kaggle Dataset)

```
# Attach your cropped DEM patches as a Kaggle Dataset first (Add Input), then:
DEM_SRC = "/kaggle/input/lunar-dem-patches-v1" # <- your dataset slug
dem_files = sorted(glob.glob(f"{DEM_SRC}/**/*.tif", recursive=True))
print(f"{len(dem_files)} DEM patch files found")
assert len(dem_files) > 0, "Attach the DEM-patches dataset via Add Input before running this cell"
```

A.4.4 Cell 3 — synthetic sun-angle pair generator (Lommel-Seeliger shading)

```
def lommel_seeliger(albedo, i, e):
    return albedo * np.cos(i) / (np.cos(i) + np.cos(e) + 1e-8)

def render_shaded(dem, sun_azimuth_deg, sun_elevation_deg, albedo=0.12):
    gy, gx = np.gradient(dem)
    normal = np.dstack([-gx, -gy, np.ones_like(dem)])
    normal /= np.linalg.norm(normal, axis=2, keepdims=True)
    az, el = np.radians(sun_azimuth_deg), np.radians(sun_elevation_deg)
    sun_vec = np.array([np.cos(el)*np.cos(az), np.cos(el)*np.sin(az), np.sin(el)])
    cos_i = np.clip(normal @ sun_vec, 0, 1)
    i = np.arccos(cos_i)
    e = np.zeros_like(i)
    return lommel_seeliger(albedo, i, e).astype(np.float32)

def build_triplet(dem_path, other_dem_paths, patch=128):
    with rasterio.open(dem_path) as src:
        dem = src.read(1).astype(np.float32)
        if dem.shape[0] < patch or dem.shape[1] < patch:
            return None
        y = random.randint(0, dem.shape[0]-patch)
        x = random.randint(0, dem.shape[1]-patch)
        crop = dem[y:y+patch, x:x+patch]
        anchor = render_shaded(crop, random.uniform(0, 360), random.uniform(15, 70))
        positive = render_shaded(crop, random.uniform(0, 360), random.uniform(15, 70))
        neg_path = random.choice(other_dem_paths)
        with rasterio.open(neg_path) as src:
            neg_dem = src.read(1).astype(np.float32)
            if neg_dem.shape[0] < patch or neg_dem.shape[1] < patch:
                return None
            ny = random.randint(0, neg_dem.shape[0]-patch)
            nx = random.randint(0, neg_dem.shape[1]-patch)
```

```

neg_crop = neg_dem[ny:ny+patch, nx:nx+patch]
negative = render_shaded(neg_crop, random.uniform(0,360), random.uniform(15,70))

def norm(a):
    a = (a - a.min()) / (a.max() - a.min() + 1e-8)
    return a[None, :, :] # (1, H, W)
return norm(anchor), norm(positive), norm(negative)

class TripletDataset(Dataset):
    def __init__(self, dem_files, patch=128, length=20000):
        self.dem_files = dem_files
        self.patch = patch
        self.length = length

    def __len__(self):
        return self.length

    def __getitem__(self, idx):
        path = random.choice(self.dem_files)
        others = [p for p in self.dem_files if p != path] or self.dem_files
        triplet = None
        while triplet is None:
            triplet = build_triplet(path, others, self.patch)
        a, p, n = triplet
        return torch.from_numpy(a), torch.from_numpy(p), torch.from_numpy(n)

dataset = TripletDataset(dem_files, patch=128, length=20000)
dataloader = DataLoader(dataset, batch_size=64, shuffle=True,
                        num_workers=4, pin_memory=True, drop_last=True)

```

A.4.5 Cell 4 — model, loss, training loop (DataParallel across both T4s)

```

class PatchEncoder(nn.Module):
    def __init__(self, embed_dim=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(1, 32, 5, stride=2, padding=2), nn.BatchNorm2d(32), nn.ReLU(),
            nn.Conv2d(32, 64, 3, stride=2, padding=1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.Conv2d(64, 128, 3, stride=2, padding=1), nn.BatchNorm2d(128), nn.ReLU(),
            nn.AdaptiveAvgPool2d(1),
        )
        self.fc = nn.Linear(128, embed_dim)

    def forward(self, x):
        z = self.fc(self.net(x).flatten(1))
        return F.normalize(z, dim=1)

def triplet_loss(a, p, n, margin=0.3):
    d_pos = 1 - (a * p).sum(dim=1)
    d_neg = 1 - (a * n).sum(dim=1)
    return F.relu(d_pos - d_neg + margin).mean()

model = PatchEncoder().to(DEVICE)
if torch.cuda.device_count() > 1: # use both T4s
    model = nn.DataParallel(model)

opt = torch.optim.AdamW(model.parameters(), lr=3e-4, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=50)

EPOCHS = 50
for epoch in range(EPOCHS):
    model.train()
    running = 0.0
    for anchor, positive, negative in dataloader:
        anchor, positive, negative = (t.to(DEVICE, non_blocking=True)
                                     for t in (anchor, positive, negative))
        with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
            za, zp, zn = model(anchor), model(positive), model(negative)
            loss = triplet_loss(za, zp, zn)
        opt.zero_grad(set_to_none=True)

```

```

        loss.backward()
        opt.step()
        running += loss.item()
    sched.step()
    print(f"epoch {epoch:02d} loss {running/len(dataloader):.4f}")
    if epoch % 10 == 0:
        torch.save(
            (model.module if hasattr(model, "module") else model).state_dict(),
            f"{CKPT_DIR}/epoch{epoch}.pt"
        )

```

A.4.6 Cell 5 — validation (never assert on a fixed embedding, only on ranking)

```

@torch.no_grad()
def validate(model, n_trials=500):
    m = model.module if hasattr(model, "module") else model
    m.eval()
    correct = 0
    val_ds = TripletDataset(dem_files, patch=128, length=n_trials)
    for a, p, n in val_ds:
        a, p, n = a.unsqueeze(0).to(DEVICE), p.unsqueeze(0).to(DEVICE), n.unsqueeze(0).to(DEVICE)
        za, zp, zn = m(a), m(p), m(n)
        d_pos = 1 - (za*zp).sum()
        d_neg = 1 - (za*zn).sum()
        correct += (d_pos < d_neg).item()
    return correct / n_trials

acc = validate(model)
print(f"triplet ranking accuracy: {acc:.3f} (target: comfortably > 0.5, ideally > 0.9)")

```

A.4.7 Cell 6 — export to ONNX (the only file that carries forward)

```

m = model.module if hasattr(model, "module") else model
m.eval().to(DEVICE)
dummy = torch.randn(1, 1, 128, 128, device=DEVICE)
torch.onnx.export(
    m, dummy, "/kaggle/working/descriptor.onnx",
    input_names=["patch"], output_names=["embedding"], opset_version=17,
)
print(os.path.getsize("/kaggle/working/descriptor.onnx") / 1e6, "MB") # expect low single-digit MB

# Trim /kaggle/working before committing -- only the ONNX export needs to survive
import shutil
shutil.rmtree(CKPT_DIR, ignore_errors=True)

```

A.4.8 Turning the export into a Kaggle Dataset, then attaching it to ARC-AGI-3

```

# In the generic notebook's own terminal, or a local machine with the Kaggle CLI + descriptor.onnx:
mkdir -p descriptor_export
cp /kaggle/working/descriptor.onnx descriptor_export/
cd descriptor_export

kaggle datasets init -p .
# edit dataset-metadata.json: set "title" and "id": "<your-kaggle-username>/lunar-descriptor-v1"
kaggle datasets create -p .

# subsequent updates to the same weights:
# kaggle datasets version -p . -m "retrained, epoch 50, val acc 0.93"

```

Then, inside the ARC-AGI-3 competition notebook: Add Input → Datasets → search your dataset id (lunar-descriptor-v1) → Add. It mounts read-only at `/kaggle/input/lunar-descriptor-v1/descriptor.onnx`, loadable with the same `onnxruntime.InferenceSession(...)` call shown in §C.4 below — no retraining, no internet needed inside that session. This is the same data-prep-vs-accelerated-compute split as Path 1, just with the training half moved to a separate notebook instead of the same one.

A.5 Which path to use

Both paths are valid. Use whichever fits the model size:

- **Path 2 (generic T4×2/P100 notebook) — default for this project.** The PatchEncoder triplet-loss model is small, with patch-sized inputs and ~128-D embeddings; T4×2 trains it in minutes, and staying outside the competition scope avoids burning any rtx6000 quota on iteration.
- **Path 1 (train inside the ARC-AGI-3 notebook, rtx6000 accelerator) — if you ever move to a larger backbone** where the RTX 6000's 96 GB VRAM actually matters. The mechanics are identical: data-prep pass with internet on, then switch the same notebook to the accelerator.

A.6 Keeping every output under Kaggle's 20 GB cap

Kaggle enforces a 20 GB cap per private dataset (and a separate 20 GB cap on a notebook's own /kaggle/working output at commit time). Nothing in this project's pipeline is naturally anywhere near that limit if the crop-before-upload discipline below is followed — the actual bottleneck is almost always someone uploading a full scene instead of a patch.

Artifact	Typical size	Cap risk	Rule
Raw PRADAN OHRC/TMC-2 .img scene	1–8 GB per scene	High if whole scenes are uploaded	Never uploaded as a dataset — stays local/PRADAN-side; only crops leave
Cropped DEM patch (128–512 px, GeoTIFF)	0.1–5 MB each	Negligible	This is what goes in the lunar-dem-patches-v1 dataset
Synthetic pair set (rendered from patches)	A few hundred MB for thousands of pairs	Low	Generate on the fly in Cell 3 rather than pre-rendering to disk, unless a fixed eval set is needed
Checkpoint .pt files (per-epoch)	~1–3 MB each (small model)	Negligible individually, adds up	Delete epoch checkpoints before committing; keep only the final export
Exported descriptor.onnx	1–5 MB	None	This is the only artifact that should ever leave as the training dataset
Crater catalog subset (CSV/shapefile)	A few MB regionally	Negligible	Filter to your scene's footprint before upload (see Document 3, Step 7)

Practical check before any `kaggle datasets create` call: `du -sh descriptor_export/`. For this project that number should read single-digit megabytes, not gigabytes — if it doesn't, something raw got copied in by mistake.

Part B — Real training infrastructure options, ranked by what's actually available to you

B.1 You (or your college lab) actually have a physical RTX 6000 Pro workstation

This is the simplest case. Steps:

- Confirm the card: `nvidia-smi` — look for RTX PRO 6000 Blackwell (workstation edition) or RTX 6000 Ada (previous generation — still very capable, just less VRAM: 48 GB vs. the Blackwell card's 96 GB).
- Install a CUDA driver matching your PyTorch build. For a Blackwell-class card you need a driver supporting CUDA 12.6+ (ideally 12.8/13.x once available).
- Create an isolated environment and install the training stack.
- Verify GPU visibility with a quick torch import.

```
conda create -n lunar python=3.11 -y
conda activate lunar
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
pip install opencv-python rasterio gdal pds4-tools spectral \
    onnx onnxruntime-gpu pytorch-lightning tqdm

python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

B.2 You don't own one — rent one by the hour

This is the realistic path for most hackathon teams and is genuinely cheap for a 7–10 day sprint, roughly cheapest/simplest to set up first:

- **RunPod (runpod.io)** — on-demand community-cloud GPU pods, RTX 6000 Ada/Blackwell-class instances by the hour, pay-as-you-go, PyTorch-preloaded templates.
- **Vast.ai** — a GPU marketplace, often the cheapest per-hour RTX 6000-class pricing, more manual setup.
- **Lambda Labs / Paperspace** — similar on-demand rental, slightly more polished UI, marginally pricier.
- **Google Colab Pro+ or Kaggle's free generic notebooks (B.3)** if budget is the binding constraint and an A100/T4-class card is an acceptable substitute for early iteration, saving rented RTX 6000 hours for the final full training run.

Workflow once rented: SSH in, run the same B.1 setup script, `rsync/scp` the prepared dataset up, train, then `scp` the exported `.onnx` file back down — the FastAPI backend does not need to run on the rented box; train there, serve locally/on Supabase-adjacent infra.

B.3 Budget-zero fallback: generic (non-ARC-AGI-3) Kaggle or Colab notebooks

This is now the same mechanism formalized as Path 2 in A.4 above: use these for architecture debugging and the synthetic-pair training loop while B.1/B.2 are arranged for a larger final run, if one turns out to be needed.

```
# In a standard Kaggle notebook (Settings -> Accelerator -> GPU T4 x2), internet ON:
!pip install -q onnxruntime rasterio pds4-tools
```

Part C — What the descriptor model actually does in your pipeline

Per the repo layout (§12 of the original PDF), this model lives at `src/matching/` and is invoked from the FastAPI `/match` endpoint as one candidate similarity source alongside classical SIFT, feeding the same downstream RANSAC geometric verification (Stage 6) — it does not replace the closed-form geometry anywhere in the pipeline.

Input: two fixed-size image patches (e.g., 128×128), one cropped from the Chandrayaan-2 source image (OHRC/TMC-2/IIRS-band-projected), one cropped from the candidate LRO NAC/SELENE reference region.

Output: a fixed-length embedding vector per patch (e.g., 128-D); the comparison result is the Euclidean or cosine distance between the two embeddings — a small distance means “likely the same ground feature despite illumination/scale differences,” a large distance means “not a match.” This distance is what gets thresholded to produce candidate correspondences before RANSAC.

The comparison method, precisely: a Siamese (twin-tower) convolutional network — the same weights applied to both patches independently, trained with a contrastive/triplet loss so that embeddings of true-matching patches (same ground, different sun angle) end up close together in embedding space, and embeddings of non-matching patches end up far apart. This is what makes it robust to the illumination variation that breaks raw-intensity SIFT matching — it's not memorizing lunar imagery, it's learning a distance function that's invariant to the specific transform (shadow-casting under varying sun elevation) it was trained against.

C.1 Data preparation

```
# generate_synthetic_pairs.py -- builds (anchor, positive, negative) triplets from a DEM patch
import numpy as np
import rasterio

def lommel_seeliger(albedo, i, e):
    """Forward-render brightness at incidence angle i, emission angle e (radians)."""
    return albedo * (np.cos(i)) / (np.cos(i) + np.cos(e) + 1e-8)

def render_shaded(dem, sun_azimuth_deg, sun_elevation_deg, albedo=0.12):
    gy, gx = np.gradient(dem)
    normal = np.dstack([-gx, -gy, np.ones_like(dem)])
    normal /= np.linalg.norm(normal, axis=2, keepdims=True)
    az, el = np.radians(sun_azimuth_deg), np.radians(sun_elevation_deg)
    sun_vec = np.array([np.cos(el)*np.cos(az), np.cos(el)*np.sin(az), np.sin(el)])
    cos_i = np.clip(normal @ sun_vec, 0, 1)
    i = np.arccos(cos_i)
    e = np.zeros_like(i) # nadir-viewing approximation for the synthetic pair
    return lommel_seeliger(albedo, i, e)

with rasterio.open("data/dem/patch_001.tif") as src:
    dem = src.read(1).astype(np.float32)

anchor = render_shaded(dem, sun_azimuth_deg=110, sun_elevation_deg=25)
positive = render_shaded(dem, sun_azimuth_deg=140, sun_elevation_deg=55) # same ground, different sun
# negative: a *different* DEM patch, any sun angle
```

C.2 Model definition and training loop (reference copy)

This is the same PatchEncoder / triplet_loss shown in full in A.3 (Cell 2) and A.4.5 (Cell 4) above — kept here as the canonical, path-independent definition that both training paths implement identically.

C.3 Validation — per your own PDF's testing strategy

Never assert against one fixed output vector (there is no single correct embedding). Instead, validate on ranking accuracy ($d_{\text{pos}} < d_{\text{neg}}$), exactly as shown in A.4.6's Cell 5 above; target > 0.5 comfortably, ideally > 0.9 .

C.4 Export to ONNX and serve — never GGUF, per your own spec

```
model.eval()
dummy = torch.randn(1, 1, 128, 128, device="cuda")
torch.onnx.export(model, dummy, "models/descriptor.onnx",
    input_names=["patch"], output_names=["embedding"], opset_version=17)

# inside FastAPI's /match handler
import onnxruntime as ort
session = ort.InferenceSession(
    "models/descriptor.onnx",
    providers=["CUDAExecutionProvider", "CPUExecutionProvider"],
)

def embed(patch: np.ndarray) -> np.ndarray:
    return session.run(["embedding"], {"patch": patch[None, None, :, :].astype(np.float32)})[0]
```

C.5 Where it plugs into the FastAPI route surface

```
POST /match
body: { scene_id, candidate_patch_pairs: [...] }

1. classical branch: SIFT descriptor + ratio test -> candidate correspondences
2. learned branch: PatchEncoder embedding distance -> candidate correspondences
3. union both candidate sets, hand to /verify (RANSAC + homography, Log 03 Problem 2)

returns: { correspondences, per-branch counts, method_breakdown }
```

Reporting both branches' contributions separately in your metrics table is itself a stronger result than reporting one merged number — it's the direct, honest version of the “baseline failing → your method holding” demo beat from the pitch plan (Document 6).